

I will be sharing you code and problem statement of experiment and you have to give me complete theory then algo in a way i can write complete code just remembering the algorithm, then explain each line of code properly in simplest possible way and in detail and then give 40-50 viva questions with answers

Perfect. Send me for each experiment:

1. **Problem Statement**
2. **Code**
3. (Optional) Your syllabus topic or experiment number

And for every experiment, I will give you:

What You'll Get

1. Complete Theory

- Introduction
- Definition
- Real-world use
- Working concept
- Key terms
- Advantages
- Disadvantages
- Applications
- Time Complexity
- Space Complexity
- Completeness
- Optimality (if applicable)

2. Algorithm (Exam-Writing Style)

I'll write the algorithm:

- Step-by-step
- Simple English
- Easy to memorize
- In practical notebook format

So you can reproduce the code just by remembering the algorithm.

3. Full Code Explanation

I'll explain:

- Every line
- Every variable
- Every loop
- Every condition
- Why that line is used
- What happens internally

In the simplest possible language.

4. Dry Run / Execution Trace

I'll show:

- Input
- Step-by-step execution
- Output formation

So you can explain it confidently in viva.

5. Important Viva Questions with Answers

You'll get:

- 40–50 viva questions
- Short answers
- Long answers
- Tricky external examiner questions
- “Why” based questions
- Comparison questions
- Scenario-based questions

6. Extra Viva Preparation

I'll also include:

- Common mistakes
- Frequently asked external questions
- Memory tricks
- Important points to speak confidently

7. Notebook-Friendly Format

Everything will be formatted in a way that you can:

- Write in practical journals
- Study before viva
- Revise quickly

Send the first experiment whenever you're ready.



1. Implement depth first search algorithm and Breadth First Search algorithm. Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure.

AI_EXP_1.py
Python

Lets start with experiment no - 1

I have the problem statement for Experiment 1:

Implement Depth First Search (DFS) and Breadth First Search (BFS) using an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree.

But the uploaded Python code file has expired from the session storage, so I can't access the code right now.

Please upload the Experiment 1 code file again (or paste the code here).

Once you do that, I'll give you:

- Complete theory of DFS and BFS
- Graph representation explanation
- Recursive DFS explanation
- Algorithm in practical-writing format
- Full line-by-line code explanation
- Dry run with example graph
- Time & space complexity
- Differences between DFS and BFS
- Applications
- 40–50 viva questions with answers
- Tricky external viva questions too

Upload the code and we'll start properly.



AI_EXP_1.py
Python

Stopped thinking >

Experiment No. 1

Depth First Search (DFS) and Breadth First Search (BFS)

Problem Statement

Implement Depth First Search (DFS) algorithm and Breadth First Search (BFS) algorithm. Use an undirected graph and develop a recursive algorithm for searching all the vertices of a graph or tree data structure.

1. THEORY

Introduction to Graph

A graph is a non-linear data structure consisting of:

- **Vertices (Nodes)** → Represent entities
- **Edges** → Connections between vertices

Example:

- Cities connected by roads
- Users connected in social networks
- Web pages connected through links

Types of Graph

1. Directed Graph

Edges have direction.

Example:

A → B

2. Undirected Graph

Edges do not have direction.

Example:

A — B

In this experiment, we use an **undirected graph**.

Graph Representation

Adjacency List

In adjacency list representation:

- Every vertex stores its neighboring vertices.

Example:

```
graph = {
    0: [1,2],
    1: [0,3],
    2: [0,3],
    3: [1,2]
}
```

PythonRun

Meaning:

- Vertex 0 is connected to 1 and 2
- Vertex 1 is connected to 0 and 3

What is Graph Traversal?

Traversal means visiting all vertices of a graph systematically.

Two important traversal methods:

1. DFS
2. BFS

Depth First Search (DFS)

DFS visits:

- One path completely first
- Then backtracks

It works like:

“Go as deep as possible first.”

Working of DFS

Suppose graph:



Starting from 0:

- Visit 0
- Go to 1
- Go to 3
- Go to 2

DFS Order:

0 → 1 → 3 → 2

Characteristics of DFS

- Uses recursion or stack
- Goes deep first
- Backtracking is used

Applications of DFS

1. Path finding
2. Cycle detection
3. Topological sorting
4. Solving mazes
5. Connected components

Advantages of DFS

- Requires less memory
- Easy recursive implementation
- Useful for deep solutions

Disadvantages of DFS

- May not give shortest path
- Can go infinitely deep in cyclic graphs if visited nodes are not tracked

Breadth First Search (BFS)

BFS visits:

- All neighboring nodes first
- Then moves to next level

It works like:

“Visit level by level.”

Working of BFS

Starting from 0:

Visit:

- 0
- Then neighbors 1 and 2
- Then 3

BFS Order:

0 → 1 → 2 → 3

Characteristics of BFS

- Uses Queue
- Level-order traversal
- Finds shortest path in unweighted graph

Applications of BFS

1. Shortest path
2. GPS navigation
3. Network broadcasting
4. Social networking
5. Web crawling

Advantages of BFS

- Finds shortest path
- Complete traversal
- Systematic searching

Disadvantages of BFS

- Requires more memory
- Queue can become large

Difference Between DFS and BFS

Feature	DFS	BFS
Data Structure	Stack / Recursion	Queue
Traversal	Depth-wise	Level-wise
Memory	Less	More
Shortest Path	No	Yes
Speed	Faster sometimes	Slower sometimes

Time Complexity

For both DFS and BFS:

Time Complexity

$$O(V + E)$$

Where:

- V = Number of vertices
- E = Number of edges

Space Complexity

DFS

$O(V)$

BFS

$O(V)$

Completeness and Optimality

Algorithm	Complete	Optimal
DFS	No	No
BFS	Yes	Yes (for unweighted graph)

2. ALGORITHM

Algorithm for DFS

Step-by-Step Algorithm

1. Start
2. Create a visited set
3. Visit starting vertex
4. Mark it as visited
5. Print the vertex
6. Traverse all neighboring vertices
7. If neighbor is not visited:
 - Recursively call DFS
8. Stop when all vertices are visited

Easy Memory Trick for DFS

Visit → Mark → Go Deep → Backtrack

Algorithm for BFS

1. Start
2. Create visited set
3. Create queue
4. Insert starting vertex into queue
5. Mark starting vertex visited
6. Repeat until queue becomes empty:
 - Remove front vertex
 - Print vertex
 - Visit all neighbors
 - If neighbor not visited:
 - Mark visited
 - Insert into queue
7. Stop

Easy Memory Trick for BFS

Queue → Visit → Add Neighbors → Repeat

3. CODE

```
from collections import deque
```

```
# DFS (Recursive)
def dfs(vertex, graph, visited):
    visited.add(vertex)
    print(vertex," ")

    for neighbour in graph[vertex]:
        if neighbour not in visited:
            dfs(neighbour, graph, visited)

# BFS (Using Queue)
def bfs(start, graph):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex," ")

        for neighbour in graph[vertex]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

# Graph (Adjacency List)
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
    3: [1, 2]
}

print("DFS Traversal:")
dfs(0, graph, set())
```

Python

Run

4. LINE BY LINE EXPLANATION

Import Statement

Python

Run

Meaning

Imports `deque` from `collections` module.

Why?

Deque works as an efficient queue for BFS.

DFS FUNCTION

Function Definition

Python

Run

Meaning

Creates DFS function.

Parameters

- vertex → current node
- graph → adjacency list
- visited → stores visited nodes

Mark Vertex as Visited

Python

Run

Meaning

Adds current node to visited set.

Why?

To avoid visiting same node again.

Print Vertex

Python

Run

Displays current node.

Loop Through Neighbors

Python

Run

Access all connected nodes.

Check if Neighbor Already Visited

Python

Run

Prevents infinite looping.

Recursive DFS Call

Python

Run

Calls DFS again for neighboring node.

This is the main recursion step.

BFS FUNCTION

Function Definition

Python

Run

Creates BFS function.

Create Visited Set

Python

Run

Stores visited nodes.

Create Queue

Python

Run

Adds starting node to queue.

Mark Start Node Visited

Python

Run

Avoid revisiting.

While Queue Not Empty

Python

Run

Loop continues until queue becomes empty.

Remove Front Element

Python

Run

Removes first element from queue.

FIFO principle:
First In First Out.

Print Vertex

Python

Run

Displays current node.

Traverse Neighbors

Python

Run

Checks connected nodes.

If Neighbor Not Visited

Python

Run

Avoids duplicates.

Mark Visited

Python

Run

Marks node visited.

Add Neighbor to Queue

Python

Stores node for future traversal.

Run

GRAPH CREATION

```
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0, 3],
}
```

Python

Adjacency list representation.

Run

DFS CALL

Python

Starts DFS from node 0.

Run

BFS CALL

Python

Starts BFS from node 0.

Run

5. DRY RUN

DFS Dry Run

Start = 0

Visited:

0 → 1 → 3 → 2

Output:

DFS Traversal:
0 1 3 2

BFS Dry Run

Queue Process:

Queue	Output
0	0
1,2	1
2,3	2
3	3

Output:

BFS Traversal:
0 1 2 3

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is a graph?

A graph is a collection of vertices and edges.

2. What is graph traversal?

Process of visiting all vertices systematically.

3. Name graph traversal algorithms.

DFS and BFS.

4. What is DFS?

Depth First Search explores deep paths first.

5. What is BFS?

Breadth First Search explores level by level.

6. Which data structure is used in DFS?

Stack or recursion.

7. Which data structure is used in BFS?

Queue.

8. Why is recursion used in DFS?

Because recursion internally uses stack.

9. Why do we use visited set?

To avoid revisiting nodes.

10. What happens if visited set is not used?

Infinite loop may occur.

Intermediate Questions

11. What is adjacency list?

Graph representation storing neighbors of each node.

12. Which traversal finds shortest path?

BFS.

13. Which traversal uses more memory?

BFS.

14. Why?

Because it stores entire level in queue.

15. Can DFS find shortest path?

Not guaranteed.

16. What is backtracking in DFS?

Returning to previous node after exploring deep path.

17. What is FIFO?

First In First Out.

18. What is LIFO?

Last In First Out.

19. Which traversal uses FIFO?

BFS.

20. Which traversal uses LIFO?

DFS.

Advanced Questions

21. Time complexity of DFS?

$O(V + E)$

22. Time complexity of BFS?

$O(V + E)$

23. Space complexity of DFS?

$O(V)$

24. Space complexity of BFS?

$O(V)$

25. Is BFS complete?

Yes.

26. Is DFS complete?

No.

27. Is BFS optimal?

Yes for unweighted graphs.

28. Is DFS optimal?

No.

29. Real-life application of BFS?

GPS shortest route.

30. Real-life application of DFS?

Maze solving.

Tricky External Viva Questions

31. Why is BFS better for shortest path?

Because it explores nearest nodes first.

32. Why is DFS memory efficient?

Because it stores only current path.

33. Can BFS be recursive?

Possible but uncommon.

34. Can DFS be iterative?

Yes using stack.

35. What is cyclic graph?

Graph containing cycles.

36. What is disconnected graph?

Graph where some vertices are unreachable.

37. Can DFS work on tree?

Yes.

38. Difference between tree and graph?

Tree has no cycles.

39. What is recursion?

Function calling itself.

40. Why deque is preferred in BFS?

Efficient insertion and deletion.

Very Important “WHY” Questions

41. Why queue is used in BFS?

To process nodes level-wise.

42. Why stack is used in DFS?

To remember previous nodes for backtracking.

43. Why BFS guarantees shortest path?

Because nearest nodes are visited first.

44. Why recursion simplifies DFS?

Automatic stack handling.

45. Why adjacency list is memory efficient?

Stores only existing edges.

Output Questions

46. What is DFS output for given graph?

0 1 3 2

47. What is BFS output?

0 1 2 3

48. Why outputs differ?

Traversal strategies differ.

49. Which traversal is faster?

Depends on graph structure.

50. Which algorithm is preferred in AI search?

Both depending on problem.

📄 ↺ ↻ ⋮

2. Implement A star Algorithm for any game search problem.

AI_EXP_2.py
Python

lets start with exp - 2

I have the Experiment 2 problem statement:

Implement A* (A Star) Algorithm for any game search problem.

But the uploaded code file for Experiment 2 is not accessible from the current session storage, so I can't read the program yet.

Please upload the AI_EXP_2.py file again (or paste the code here).

Once you upload it, I'll give you:

- Complete theory of A* Algorithm
- Heuristic function explanation
- $f(n) = g(n) + h(n)$ explanation
- Working of OPEN and CLOSED list
- Algorithm in easy practical-writing format
- Full line-by-line code explanation
- Dry run with state expansion
- Time & space complexity
- Completeness & optimality
- Difference between A*, BFS, DFS, Greedy Search
- Real-life applications
- 40–50 viva questions with answers
- Tricky external viva questions too

Upload the code and we'll start Experiment 2 properly.



AI_EXP_2.py
Python

Experiment No. 2

A* (A Star) Algorithm for Game Search Problem

Problem Statement

Implement A* (A Star) Algorithm for any game search problem.

1. THEORY

Introduction to A* Algorithm

A* (A Star) is one of the most important and efficient searching algorithms used in Artificial Intelligence.

It is an informed search algorithm because:

- It uses knowledge (heuristic)
- It intelligently selects the best path

A* finds:

- Shortest path
- Optimal solution
- Fastest solution

What is a Search Problem?

A search problem means:

- Finding a path from initial state to goal state.

Examples:

- Puzzle solving
- Maze solving
- GPS navigation
- Robot movement
- Game AI

What is A* Search?

A* combines:

1. Uniform Cost Search
2. Greedy Best First Search

It selects the node having:

- Lowest total estimated cost.

Formula Used in A*

The main formula is:

$$f(n) = g(n) + h(n)$$

Where:

Term	Meaning
f(n)	Total estimated cost
g(n)	Actual cost from start node
h(n)	Heuristic estimated cost to goal

Meaning of Formula

$g(n)$

Cost already travelled.

Example:

- Number of moves already performed.

$h(n)$

Estimated distance from current state to goal.

In this experiment:

- Misplaced tiles count is used.

$f(n)$

Total estimated solution cost.

A* always selects node having:

- Smallest $f(n)$

Puzzle Problem Used

We use:

8 Puzzle Problem

Puzzle contains:

- Numbers from 1 to 8
- One blank space (0)

Goal:

Arrange tiles in correct order.

Goal State

1	2	3
4	5	6
7	8	0

0 represents blank tile.

Initial State

1	2	3
5	6	0
7	8	4

Heuristic Function

Misplaced Tile Heuristic

Counts how many tiles are in wrong position.

Example:

1	2	3
5	6	0
7	8	4

Wrong tiles:

- 5
- 6
- 4

So:

$$h(n) = 3$$

OPEN List and CLOSED List

OPEN List

Contains nodes to be explored.

Implemented using:

- Priority Queue

CLOSED List

Contains already visited nodes.

Implemented using:

- Set

Working of A*

1. Start from initial state
2. Calculate heuristic
3. Insert into priority queue
4. Remove node with smallest $f(n)$
5. Check goal state
6. Generate child states
7. Calculate $f(n)$
8. Repeat until goal found

Characteristics of A*

Feature	Description
Type	Informed Search
Uses Heuristic	Yes
Complete	Yes
Optimal	Yes
Data Structure	Priority Queue

Advantages of A*

1. Fast searching
2. Finds optimal path
3. Intelligent traversal
4. Widely used in AI

Disadvantages of A*

1. Uses more memory
2. Slower for huge state space
3. Depends on heuristic quality

Applications of A*

1. GPS Navigation
2. Robotics
3. Video games

- 4. Puzzle solving
- 5. Path planning
- 6. AI agents

Time Complexity

Worst case:

$O(b^d)$

Where:

- b = branching factor
- d = depth of solution

Space Complexity

$O(b^d)$

A* stores many nodes in memory.

Completeness and Optimality

Property	A*
Complete	Yes
Optimal	Yes

If heuristic is admissible.

What is Admissible Heuristic?

A heuristic that:

- Never overestimates actual cost.

Misplaced tile heuristic is admissible.

Difference Between A* and Other Algorithms

Algorithm	Uses Heuristic	Optimal
DFS	No	No
BFS	No	Yes
Greedy Search	Yes	No
A*	Yes	Yes

2. ALGORITHM

A* Algorithm

Step-by-Step Algorithm

1. Start
2. Create OPEN list using priority queue
3. Create CLOSED list
4. Insert initial state into OPEN
5. Repeat until OPEN becomes empty:
 - Remove node with smallest f(n)
 - If goal found:
 - Stop
 - Mark node visited
 - Generate all possible child states
 - Calculate g(n)

- Calculate h(n)
 - Calculate f(n)
 - Insert into OPEN list
6. If no solution:
- Print solution not found
7. Stop

Easy Memory Trick

Choose Minimum f(n)
Expand
Generate Children
Repeat

3. CODE

```
import heapq

# it means the size of the puzzle is 3x3
N = 3

# Goal state
goal = [[1,2,3],
        [4,5,6],
        [7,8,0]]

# Heuristic: count misplaced tiles
def calculate_heuristic(mat):
    count = 0
    for i in range(N):
        for j in range(N):
            if mat[i][j] != 0 and mat[i][j] != goal[i][j]:
                count += 1
    return count

# Check goal state
def is_goal(mat):
    return mat == goal

# A* Algorithm
def solve_astar(start, x, y):
    visited = set()

    # priority queue -> (f(n), g(n), matrix, x, y)
    pq = []

    h = calculate_heuristic(start)
    heapq.heappush(pq, (h, 0, start, x, y))

    # Moves: Down, Left, Up, Right
    row = [1, 0, -1, 0]
    col = [0, -1, 0, 1]

    while pq:
        f, cost, mat, x, y = heapq.heappop(pq)

        # Convert matrix to tuple for set
        mat_tuple = tuple(tuple(row) for row in mat)

        if mat_tuple in visited:
            continue

        visited.add(mat_tuple)

        if is_goal(mat):
            print("Goal State Reached!")
            return

        # Try all 4 moves
        for i in range(4):
            newX = x + row[i]
            newY = y + col[i]

            if 0 <= newX < N and 0 <= newY < N:
                new_mat = [row[:] for row in mat]

                # Swap blank
                new_mat[x][y], new_mat[newX][newY] = new_mat[newX][newY], new_mat[x][y]
```

```
new_tuple = tuple(tuple(r) for r in new_mat)

if new_tuple not in visited:
    g = cost + 1
    h = calculate_heuristic(new_mat)
    heapq.heappush(pq, (g + h, g, new_mat, newX, newY))

print("Solution Not Found!")

# Driver Code
start = [[1, 2, 3],
         [5, 6, 0],
         [7, 8, 4]]
```

PythonRun

4. LINE BY LINE EXPLANATION

Import Module

PythonRun

Imports priority queue module.

Used for:

- Getting minimum f(n) state quickly.

Puzzle Size

PythonRun

Defines:

- 3×3 puzzle size.

Goal State

```
goal = [[1,2,3],
```

PythonRun

Final desired arrangement.

Heuristic Function

PythonRun

Creates heuristic function.

Initialize Count

PythonRun

Stores misplaced tile count.

Traverse Matrix

Python

Checks every tile.

Run

Check Wrong Position

Python

Conditions:

- Ignore blank tile
- Check wrong position

Run

Increase Count

Python

Increase heuristic value.

Run

Return Heuristic

Python

Returns misplaced tile count.

Run

Goal Check Function

Python

Checks if puzzle solved.

Run

A* Function

Python

Parameters:

- start → initial puzzle
- x,y → blank tile position

Run

Create Visited Set

Python

Stores explored states.

Run

Create Priority Queue

Python

Run

Stores nodes according to priority.

Calculate Initial Heuristic

Python

Run

Find heuristic of start state.

Insert into Queue

Python

Run

Stores:

`(f(n), g(n), matrix, x, y)`

Initially:

`f = h`
`g = 0`

Movement Arrays

Python

Run

Represents:

- Down
- Left
- Up
- Right

Main Loop

Python

Run

Runs until queue becomes empty.

Remove Best Node

Python

Run

Removes state having:

- Minimum $f(n)$

Convert Matrix to Tuple

Python

Run

Why?

Lists cannot be stored in set.

Tuple is immutable.

Already Visited Check

Python	Run
Avoid repeated states.	

Mark Visited

Python	Run
Adds current state.	

Goal Check

Python	Run
Checks solution.	

Print Goal Message

Python	Run
Displays success.	

Generate Moves

Python	Run
Try all four directions.	

New Coordinates

Python	Run
Find new blank position.	

Boundary Check

Python	Run
Ensures valid move.	

Copy Matrix

Python	Run
--------	-----

Creates deep copy.

Swap Tiles

Python

Run

Moves blank tile.

New State Tuple

Python

Run

Convert for set checking.

Check Not Visited

Python

Run

Avoid duplicates.

Calculate New Cost

Python

Run

Each move costs 1.

Calculate Heuristic

Python

Run

Find heuristic.

Push into Queue

Python

Run

Insert new state.

Solution Failure

Python

Run

If no solution exists.

Driver Code

```
start = [[1, 2, 3],
```

Python

Run

Initial puzzle.

Blank Position

Blank tile coordinates.

Function Call

Starts algorithm.

5. DRY RUN

Initial State

```
1 2 3
5 6 0
7 8 4
```

Heuristic Calculation

Wrong tiles:

- 5
- 6
- 4

So:

$$h(n) = 3$$

Initial:

- g(n)=0
- f(n)=3

First Expansion

Blank moves:

- Left
- Down

New states generated.

A* chooses state with:

- Minimum f(n)

Goal State Reached

```
1 2 3
4 5 6
7 8 0
```

Output

Goal State Reached!

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is A* Algorithm?

A heuristic search algorithm used to find shortest path.

2. Why is it called informed search?

Because it uses heuristic knowledge.

3. What is heuristic?

Estimated cost to reach goal.

4. What is f(n)?

Total estimated cost.

5. Formula of A*?

$$f(n) = g(n) + h(n)$$

6. What is g(n)?

Actual path cost.

7. What is h(n)?

Estimated remaining cost.

8. What is OPEN list?

List of nodes to explore.

9. What is CLOSED list?

Visited nodes list.

10. Which data structure is used?

Priority Queue.

Intermediate Questions

11. Why priority queue is used?

To get minimum f(n) efficiently.

12. What is admissible heuristic?

Heuristic that never overestimates.

13. Is misplaced tile heuristic admissible?

Yes.

14. What is goal state?

Final desired puzzle arrangement.

15. What is state space?

All possible puzzle configurations.

16. What is branching factor?

Number of child states.

17. What is optimal solution?

Best minimum-cost solution.

18. Is A* complete?

Yes.

19. Is A* optimal?

Yes.

20. Why is A* better than DFS?

It intelligently selects best path.

Advanced Questions

21. Time complexity of A*?

$O(b^d)$

22. Space complexity?

$O(b^d)$

23. Why does A* consume more memory?

Stores many states.

24. What happens with poor heuristic?

Performance decreases.

25. Difference between BFS and A*?

A* uses heuristic.

26. Difference between Greedy Search and A*?

Greedy uses only $h(n)$, A* uses $g(n)+h(n)$.

27. Can A* fail?

Yes if heuristic is poor or state space huge.

28. Why tuples are used in set?

Lists are mutable.

29. Why deep copy is needed?

To avoid modifying original matrix.

30. Why boundary check is important?

Prevent invalid moves.

Tricky External Viva Questions

31. Why blank tile represented by 0?

Easy identification.

32. Why recursion not used here?

Priority queue approach is easier.

33. What if heuristic becomes zero?

A* behaves like Uniform Cost Search.

34. What if $g(n)=0$ always?

A* becomes Greedy Best First Search.

35. Why heuristic should not overestimate?

To maintain optimality.

36. Can multiple heuristics be used?

Yes.

37. Which heuristic is better than misplaced tiles?

Manhattan Distance.

38. What is local optimum?

Best nearby solution, not global best.

39. Can A* work on maps?

Yes.

40. Application in games?

NPC movement and pathfinding.

Important “WHY” Questions

41. Why is A* efficient?

Because it intelligently chooses promising states.

42. Why does A* use both $g(n)$ and $h(n)$?

To balance actual and estimated cost.

43. Why priority queue gives efficiency?

Fast retrieval of minimum element.

44. Why visited set is necessary?

Avoid repeated exploration.

45. Why heuristic improves searching?

Reduces unnecessary exploration.

Output Questions

46. What is output of program?

Goal State Reached!

47. What happens if no solution exists?

Prints Solution Not Found.

48. Why is heuristic initially 3?

Three misplaced tiles.

49. Which move is selected first?

Move with minimum f(n).

50. Why A* is popular in AI?

Fast and optimal searching.

📄 ↺ ↻ ⋮

3. Implement Greedy search algorithm for any of the following application:
I. Selection Sort
II. Minimum Spanning Tree
III. Single-Source Shortest Path Problem
IV. Job Scheduling Problem
V. Prim's Minimal Spanning Tree Algorithm
VI. Kruskal's Minimal Spanning Tree Algorithm
VII. Dijkstra's Minimal Spanning Tree Algorithm

For Experiment 3, your syllabus includes multiple Greedy Algorithm applications:

1. Selection Sort
2. Minimum Spanning Tree
3. Single Source Shortest Path
4. Job Scheduling Problem
5. Prim’s Algorithm
6. Kruskal’s Algorithm
7. Dijkstra’s Algorithm

To give you:

- complete theory,
- algorithms,
- detailed line-by-line explanation,
- dry runs,
- complexities,
- and 40–50 viva questions,

I need the specific code(s) you were given in your practical file.

Please upload the Experiment 3 Python code file(s) or paste the code here.
After that, I'll prepare detailed notes exactly matching your implementation.

📄 ↺ ↻ ⋮

- P3(Kruskal).py

Python
- P3(Prims).py

Python
- P3(Selection_Sort).py

Python
- P3(Dijkstras).py

Python

Lets go with one experiment at a time

Perfect. We'll go one by one.

You uploaded:

- Kruskal's Algorithm
- Prim's Algorithm
- Selection Sort
- Dijkstra's Algorithm

Let's start with the easiest and most important viva topic first:

Experiment 3 (I) — Selection Sort using Greedy Approach

Please confirm by sending:

- "Start Selection Sort"

Then I'll give:

- Complete theory of Greedy Algorithm
- Why Selection Sort is Greedy
- Step-by-step algorithm
- Full code explanation line by line
- Dry run
- Time & space complexity
- Advantages/disadvantages
- 40–50 viva questions with answers
- Tricky external viva questions
- Memory tricks for writing code in practical exam



start selection sort

The uploaded Selection Sort file is no longer accessible in the current session storage, so I can't read the exact code implementation right now.

Please upload the file again:

- P3(Selection_Sort).py

Once you upload it, I'll give you the complete detailed practical-style explanation:

- Greedy theory
- Why Selection Sort is Greedy
- Algorithm
- Line-by-line code explanation
- Dry run
- Time & space complexity
- Advantages/disadvantages
- 40–50 viva questions with answers
- External viva questions and tricks



- P3(Selection_Sort).py

Python

Experiment No. 3 (I)

Selection Sort using Greedy Approach

Problem Statement

Implement Greedy Search Algorithm using Selection Sort.

1. THEORY

Introduction to Greedy Algorithm

A Greedy Algorithm is an algorithmic approach that:

- Selects the best possible option at the current step
- Makes locally optimal choices
- Hopes to achieve global optimum solution

It does not reconsider previous choices.

Real Life Example of Greedy Method

Suppose you want minimum number of currency notes for ₹270.

Greedy choice:

- Take biggest note first.

$$200 + 50 + 20$$

At every step:

- Best immediate choice is selected.

This is greedy approach.

What is Selection Sort?

Selection Sort is a simple sorting algorithm.

It repeatedly:

1. Finds smallest element
2. Places it at correct position

It divides array into:

- Sorted part
- Unsorted part

Why Selection Sort is Greedy?

Because at every iteration:

- It selects the minimum element immediately
- Without considering future consequences

That is exactly greedy behavior.

Working of Selection Sort

Example Array:

$$64 \ 25 \ 12 \ 22 \ 11$$

Pass 1

Find smallest element:

$$11$$

Swap with first element.

Array becomes:

11 25 12 22 64

Pass 2

Find smallest from remaining:

12

Swap with second element.

Array becomes:

11 12 25 22 64

Pass 3

Find smallest:

22

Array becomes:

11 12 22 25 64

Final Sorted Array

11 12 22 25 64

Characteristics of Selection Sort

Feature	Description
Type	Comparison Sort
Method	Greedy
Stable	No
In-place	Yes

Advantages of Selection Sort

1. Easy to understand
2. Simple implementation
3. Requires less memory
4. Performs fewer swaps

Disadvantages of Selection Sort

1. Slow for large data
2. Time complexity always $O(n^2)$
3. Not suitable for huge datasets

Applications of Selection Sort

1. Small datasets
2. Educational purposes
3. Embedded systems
4. Memory-constrained systems

Time Complexity

Best Case

$O(n^2)$

Average Case

$O(n^2)$

Worst Case

$O(n^2)$

Space Complexity

$O(1)$

Because sorting is done in same array.

Completeness and Optimality

Property	Selection Sort
Complete	Yes
Optimal	No

2. ALGORITHM

Selection Sort Algorithm

Step-by-Step Algorithm

1. Start
2. Traverse array from first element
3. Assume current index contains minimum value
4. Compare it with remaining elements
5. Find actual minimum element
6. Swap minimum element with current index
7. Repeat for remaining array
8. Stop when array becomes sorted

Easy Memory Trick

Find Minimum → Swap → Repeat

3. CODE

```
# Selection Sort using Greedy Approach

arr = [64, 25, 12, 22, 11]

n = len(arr)

for i in range(n):

    min_index = i

    for j in range(i + 1, n):

        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]
```

Python

Run

4. LINE BY LINE EXPLANATION

Array Initialization

Python

Run

Creates list of unsorted numbers.

Length of Array

Python

Run

Stores total number of elements.

Here:

```
n = 5
```

Outer Loop

Python

Run

Controls:

- Number of passes

Each pass:

- Places one element correctly.

Assume Minimum Element

Python

Run

Initially assume:

- Current element is smallest.

Inner Loop

Python

Run

Checks remaining unsorted elements.

Comparison

Python

Run

Checks whether:

- New smaller element found.

Update Minimum Index

Python

Run

Stores position of smallest element.

Swapping

Python

Run

Places smallest element at correct position.

Print Message

Python

Run

Displays heading.

Print Sorted Array

Python

Run

Displays sorted list.

5. DRY RUN

Initial Array

64 25 12 22 11

Pass 1

Minimum = 11

Swap with 64

11 25 12 22 64

Pass 2

Minimum = 12

Swap with 25

11 12 25 22 64

Pass 3

Minimum = 22

Swap with 25

11 12 22 25 64

Pass 4

Already sorted.

```
11 12 22 25 64
```

Final Output

```
Sorted Array:  
[11, 12, 22, 25, 64]
```

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is Selection Sort?

A sorting algorithm that repeatedly selects minimum element.

2. Why is Selection Sort called greedy?

Because it selects best minimum element immediately.

3. What is sorting?

Arranging data in ascending or descending order.

4. What is the main idea of Selection Sort?

Find minimum and place correctly.

5. Which type of sorting is Selection Sort?

Comparison-based sorting.

6. Is Selection Sort stable?

No.

7. Is Selection Sort in-place?

Yes.

8. What is in-place sorting?

Sorting without extra memory.

9. How many loops are used?

Two loops.

10. What is swapping?

Exchanging two elements.

Intermediate Questions

11. Time complexity of Selection Sort?

$O(n^2)$

12. Space complexity?

$O(1)$

13. Best case complexity?

$O(n^2)$

14. Worst case complexity?

$O(n^2)$

15. Why best case still $O(n^2)$?

Because comparisons are always performed.

16. What is min_index?

Stores smallest element position.

17. Why inner loop starts from $i+1$?

Previous elements are already sorted.

18. Number of passes for n elements?

$n-1$ passes.

19. Can Selection Sort sort descending order?

Yes.

20. What is comparison sort?

Sorting using comparisons.

Advanced Questions

21. Why Selection Sort is inefficient?

Too many comparisons.

22. Does Selection Sort use recursion?

No.

23. Which is better: Bubble or Selection Sort?

Selection Sort performs fewer swaps.

24. Can Selection Sort handle duplicate values?

Yes.

25. Is Selection Sort adaptive?

No.

26. What is adaptive sorting?

Sorting that performs faster for sorted arrays.

27. Can Selection Sort work on linked list?

Yes.

28. What is the maximum number of swaps?

$n-1$

29. Why fewer swaps are advantage?

Useful when swapping is costly.

30. Which data structure is mainly used?

Arrays.

Tricky External Viva Questions

31. Why is Selection Sort not stable?

Relative order of equal elements may change.

32. What if array already sorted?

Still all comparisons occur.

33. Why outer loop runs n times?

To place every element correctly.

34. Why Selection Sort is called in-place?

No extra array required.

35. Difference between Bubble and Selection Sort?

Bubble swaps repeatedly, Selection swaps once per pass.

36. Can Selection Sort sort strings?

Yes.

37. Can Selection Sort work for floating-point numbers?

Yes.

38. Which operation dominates complexity?

Comparisons.

39. Is Selection Sort divide-and-conquer?

No.

40. Is Selection Sort recursive or iterative?

Iterative.

Important “WHY” Questions

41. Why greedy approach works here?

Locally smallest element helps global sorting.

42. Why swapping needed?

To place smallest element correctly.

43. Why inner loop checks remaining array?

To find actual minimum.

44. Why Selection Sort needs less memory?

Uses same array for sorting.

45. Why complexity does not improve for sorted array?

Because all comparisons still happen.

Output Questions

46. What is output of program?

[11, 12, 22, 25, 64]

47. What is first minimum selected?

11

48. What is second minimum selected?

12

49. How many swaps performed?

4 maximum.

50. Why Selection Sort important in academics?

Easy to understand greedy concept.

📄 ↺ ↻ ⋮

P3{Dijkstras}.py
Python

Experiment No. 3 (II)

Dijkstra’s Algorithm using Greedy Approach

Problem Statement

Implement Greedy Search Algorithm for Single Source Shortest Path Problem using Dijkstra’s Algorithm.

1. THEORY

Introduction to Greedy Algorithm

A Greedy Algorithm always:

- Chooses the best possible option at current step

- Makes locally optimal choices
- Attempts to achieve globally optimal solution

It never revisits previous decisions.

What is Shortest Path Problem?

Shortest Path Problem means:

- Finding minimum distance between two vertices in a graph.

Examples:

- Google Maps
- GPS Navigation
- Internet Routing
- Network Communication

What is Dijkstra’s Algorithm?

Dijkstra’s Algorithm is a Greedy Algorithm used to:

- Find shortest distance from a source node to all other nodes in a weighted graph.

It works only when:

- Edge weights are non-negative.

Why Dijkstra is Greedy?

At every step:

- It selects the nearest unvisited node.

That immediate best choice makes it a Greedy Algorithm.

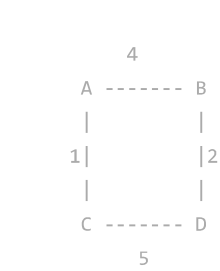
Working Principle

1. Start from source vertex
2. Assign:
 - 0 distance to source
 - Infinity to others
3. Select nearest unvisited node
4. Update neighboring distances
5. Repeat until all nodes visited

Important Terms

Term	Meaning
Vertex	Node in graph
Edge	Connection between nodes
Weight	Cost of edge
Distance Array	Stores shortest distance
Visited Array	Tracks visited nodes

Example Graph



Source = A

Shortest distances:

- $A \rightarrow C = 1$
- $A \rightarrow B = 4$
- $A \rightarrow D = 6$

Applications of Dijkstra’s Algorithm

1. GPS Navigation
2. Network Routing
3. Robot Navigation
4. Airline Route Systems
5. Communication Networks
6. Game Development

Advantages of Dijkstra’s Algorithm

1. Finds shortest path
2. Efficient for small graphs
3. Widely used
4. Greedy optimization

Disadvantages of Dijkstra’s Algorithm

1. Cannot handle negative weights
2. Slow for very large graphs
3. Uses more memory for huge graphs

Time Complexity

Using Simple Array

$O(V^2)$

Using Priority Queue

$O((V + E) \log V)$

Where:

- V = vertices
- E = edges

Space Complexity

$O(V)$

Completeness and Optimality

Property	Dijkstra
Complete	Yes
Optimal	Yes

For non-negative weights.

Difference Between BFS and Dijkstra

BFS	Dijkstra
Unweighted graph	Weighted graph
Equal cost edges	Different weights
Queue	Priority Queue

Difference Between Prim’s and Dijkstra

Prim’s	Dijkstra
Minimum Spanning Tree	Shortest Path
Connect all vertices	Minimum distance
Tree construction	Path optimization

2. ALGORITHM

Dijkstra’s Algorithm

Step-by-Step Algorithm

1. Start
2. Initialize distance array with infinity
3. Set source distance = 0
4. Mark all vertices unvisited
5. Select nearest unvisited vertex
6. Mark selected vertex visited
7. Update neighboring distances
8. Repeat until all vertices visited
9. Print shortest distances
10. Stop

Easy Memory Trick

Pick Minimum → Update Distances → Repeat

3. CODE

```
import heapq

# Dijkstra Algorithm
def dijkstra(graph, start):

    # Priority Queue
    pq = []

    # Push source node
    heapq.heappush(pq, (0, start))

    # Distance dictionary
    distances = {vertex: float('infinity') for vertex in graph}
    distances[start] = 0

    while pq:

        # Get minimum distance node
        current_distance, current_vertex = heapq.heappop(pq)

        # Skip if already larger distance
        if current_distance > distances[current_vertex]:
            continue

        # Traverse neighbors
        for neighbor, weight in graph[current_vertex]:

            distance = current_distance + weight

            # Update shortest distance
            if distance < distances[neighbor]:

                distances[neighbor] = distance

                heapq.heappush(pq, (distance, neighbor))

    return distances

# Graph Representation
```



```
graph = {
    'A': [( 'B', 4), ( 'C', 1)],
    'B': [( 'A', 4), ( 'D', 2)],
    'C': [( 'A', 1), ( 'D', 5)],
    'D': [( 'B', 2), ( 'C', 5)]
}

# Function Call
result = dijkstra(graph, 'A')

# Print Result
print("Shortest distances from source:")
```

Python

Run

4. LINE BY LINE EXPLANATION

Import Module

Python

Run

Imports priority queue module.

Used for:

- Efficient minimum distance selection.

Function Definition

Python

Run

Creates Dijkstra function.

Parameters:

- graph → adjacency list
- start → source vertex

Create Priority Queue

Python

Run

Stores vertices based on minimum distance.

Insert Source Node

Python

Run

Adds source node with distance 0.

Distance Dictionary

Python

Run

Initially:

- Every node distance = infinity.

Source Distance

Python	Run
Distance from source to itself is 0.	
Main Loop	
Python	Run
Runs until queue becomes empty.	
Remove Minimum Distance Vertex	
Python	Run
Gets nearest vertex.	
Skip Larger Distance	
Python	Run
Avoids unnecessary processing.	
Traverse Neighbors	
Python	Run
Checks connected nodes.	
Calculate New Distance	
Python	Run
Computes path cost.	
Update Condition	
Python	Run
Checks for shorter path.	
Update Distance	
Python	Run
Stores better shortest path.	
Insert into Queue	

Python

Run

Push updated node.

Return Distances

Python

Run

Returns shortest distances.

Graph Creation

```
graph = {  
    'A': [( 'B', 4), ( 'C', 1)],  
    'B': [( 'A', 4), ( 'D', 2)],  
    'C': [( 'A', 1), ( 'D', 5)],
```

Python

Run

Adjacency list representation.

Function Call

Python

Run

Starts algorithm from source A.

Print Result

Python

Run

Displays heading.

Print Distances

Python

Run

Displays shortest distance for each node.

5. DRY RUN

Initial State

Source = A

Distance Table:

Vertex	Distance
A	0
B	∞
C	∞

Vertex	Distance
D	∞

Step 1

Visit A

Update:

- B = 4
- C = 1

Step 2

Visit C

Update:

- D = 6

Step 3

Visit B

Update:

- D = min(6, 4+2)
- D = 6

Final Distances

Vertex	Distance
A	0
B	4
C	1
D	6

Output

```
Shortest distances from source:
A : 0
B : 4
C : 1
D : 6
```

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is Dijkstra’s Algorithm?

A shortest path greedy algorithm.

2. What problem does it solve?

Single Source Shortest Path Problem.

3. Why is Dijkstra greedy?

Chooses nearest node immediately.

4. What type of graph is used?

Weighted graph.

5. Can Dijkstra handle negative weights?

No.

6. What is source node?

Starting vertex.

7. What is shortest path?

Minimum cost path.

8. Which data structure is used?

Priority Queue.

9. Why priority queue is used?

Efficient minimum selection.

10. What is infinity used for?

Initial unreachable distance.

Intermediate Questions

11. Time complexity using array?

$O(V^2)$

12. Time complexity using heap?

$O((V+E)\log V)$

13. Space complexity?

$O(V)$

14. Is Dijkstra optimal?

Yes.

15. Is Dijkstra complete?

Yes.

16. What is adjacency list?

Stores neighbors of each node.

17. What is edge weight?

Cost between vertices.

18. Why source distance initialized to 0?

Distance to itself is zero.

19. Why other distances infinity?

Initially unknown paths.

20. What is relaxation?

Updating shorter distance.

Advanced Questions

21. Why negative weights not allowed?

Incorrect shortest paths may occur.

22. What happens in disconnected graph?

Some distances remain infinity.

23. Can Dijkstra work on directed graph?

Yes.

24. Can Dijkstra work on undirected graph?

Yes.

25. Difference between BFS and Dijkstra?

BFS ignores weights.

26. Difference between Prim's and Dijkstra?

Prim's builds MST, Dijkstra shortest path.

27. What is greedy choice here?

Nearest unvisited vertex.

28. What is visited node?

Already processed node.

29. Can Dijkstra find all shortest paths?

Yes from source.

30. Real life use?

GPS systems.

Tricky External Viva Questions

31. Why heapq used?

Efficient priority queue implementation.

32. Why tuple used in priority queue?

Stores distance and node together.

33. What if graph has cycle?

Algorithm still works.

34. Why continue statement used?

Avoid outdated larger distances.

35. Why dictionary used for distances?

Easy vertex mapping.

36. What if edge weights equal?

Works correctly.

37. What if source node absent?

Error may occur.

38. Can Dijkstra use matrix representation?

Yes.

39. What is path relaxation?

Improving shortest distance estimate.

40. What dominates time complexity?

Priority queue operations.

Important “WHY” Questions

41. Why greedy approach works?

Local shortest choice leads to global optimum.

42. Why nearest node selected first?

Shortest distance guaranteed.

43. Why distances updated repeatedly?

Better paths may exist.

44. Why infinity initialization important?

Represents unknown distances.

45. Why Dijkstra popular?

Efficient and reliable shortest path algorithm.

Output Questions

46. What is shortest distance from A to C?

47. What is shortest distance from A to D?

6

48. Which node visited first?

A

49. Which node visited second?

C

50. Why Dijkstra important in networking?

Used in routing protocols.

📄 ↺ ↻ ⋮

P3(Prims).py
Python

Experiment No. 3 (III)

Prim’s Algorithm using Greedy Approach

Problem Statement

Implement Greedy Search Algorithm for Minimum Spanning Tree using Prim’s Algorithm.

1. THEORY

Introduction to Greedy Algorithm

A Greedy Algorithm:

- Makes the best possible choice at current step
- Does not reconsider previous choices
- Attempts to achieve globally optimal solution

What is Minimum Spanning Tree (MST)?

A Minimum Spanning Tree is:

- A subset of graph edges
- Connects all vertices
- Has minimum total edge weight
- Contains no cycles

Conditions of MST

For a graph with:

- V vertices

MST contains:

- V - 1 edges

What is Prim’s Algorithm?

Prim’s Algorithm is a Greedy Algorithm used to:

- Find Minimum Spanning Tree of a weighted graph.

It starts from:

- Any vertex

- And repeatedly:
- Adds minimum weight edge.

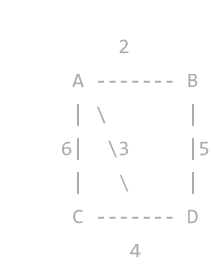
Why Prim’s Algorithm is Greedy?

- At every step:
- It selects the minimum cost edge.
- This immediate best choice makes it greedy.

Working Principle of Prim’s Algorithm

1. Start from any node
2. Mark node visited
3. Find smallest edge connecting visited to unvisited node
4. Add edge to MST
5. Repeat until all vertices visited

Example Graph



MST Construction

- Choose minimum edges:
- $A \rightarrow B = 2$
 - $A \rightarrow D = 3$
 - $D \rightarrow C = 4$

Total MST Cost:

$$2 + 3 + 4 = 9$$

Applications of Prim’s Algorithm

1. Network design
2. Cable connections
3. Road construction
4. Electrical wiring
5. Communication systems

Advantages of Prim’s Algorithm

1. Finds minimum spanning tree
2. Efficient for dense graphs
3. Simple implementation
4. Greedy optimization

Disadvantages of Prim’s Algorithm

1. Cannot handle disconnected graphs directly
2. Slightly slower for sparse graphs
3. Requires weighted graph

Characteristics of Prim’s Algorithm

Feature	Description
Algorithm Type	Greedy
Problem Solved	MST
Graph Type	Weighted Undirected
Data Structure	Priority Queue

Time Complexity

Using Adjacency Matrix

$O(V^2)$

Using Priority Queue

$O((V + E) \log V)$

Where:

- V = Vertices
- E = Edges

Space Complexity

$O(V)$

Completeness and Optimality

Property	Prim's
Complete	Yes
Optimal	Yes

Difference Between Prim's and Kruskal's

Prim's	Kruskal's
Vertex based	Edge based
Starts from node	Starts from edges
Better for dense graph	Better for sparse graph

Difference Between Prim's and Dijkstra

Prim's	Dijkstra
Minimum Spanning Tree	Shortest Path
Connects all nodes	Finds shortest distance
No source needed	Source required

2. ALGORITHM

Prim's Algorithm

Step-by-Step Algorithm

1. Start
2. Select starting vertex

3. Mark vertex visited
4. Insert connected edges into priority queue
5. Select minimum edge
6. If destination node not visited:
 - Add edge to MST
 - Mark node visited
7. Insert new connected edges
8. Repeat until all vertices visited
9. Print MST edges and total cost
10. Stop

Easy Memory Trick

Pick Minimum Edge → Visit Node → Repeat

3. CODE

```
import heapq

# Prim's Algorithm
def prim(graph, start):

    visited = set()

    # Priority Queue
    min_heap = [(0, start)]

    total_cost = 0

    while min_heap:

        weight, vertex = heapq.heappop(min_heap)

        # Skip visited vertices
        if vertex in visited:
            continue

        visited.add(vertex)

        total_cost += weight

        print("Visited:", vertex, "Weight:", weight)

        # Traverse neighbors
        for neighbor, edge_weight in graph[vertex]:

            if neighbor not in visited:
                heapq.heappush(min_heap, (edge_weight, neighbor))

    print("Total MST Cost:", total_cost)

# Graph Representation
graph = {
    'A': [('B', 2), ('C', 6), ('D', 3)],
    'B': [('A', 2), ('D', 5)],
    'C': [('A', 6), ('D', 4)],
    'D': [('A', 3), ('B', 5), ('C', 4)]
}
```

Python

Run

4. LINE BY LINE EXPLANATION

Import Module

Python

Run

Imports priority queue module.

Used for:

- Efficient minimum edge selection.

Function Definition

Python

Run

Creates Prim’s function.

Parameters:

- graph → adjacency list
- start → starting node

Create Visited Set

Python

Run

Stores visited vertices.

Create Priority Queue

Python

Run

Stores:

(weight, vertex)

Initially:

- Starting node weight = 0

Total Cost Variable

Python

Run

Stores MST cost.

Main Loop

Python

Run

Runs until queue becomes empty.

Remove Minimum Edge

Python

Run

Gets smallest edge.

Skip Visited Node

Python

Run

continue

Avoid cycle formation.

Mark Visited

Python

Run

Adds node to MST.

Add Cost

Python

Run

Updates MST total cost.

Print Visited Node

Python

Run

Displays selected edge.

Traverse Neighbors

Python

Run

Checks connected nodes.

Unvisited Neighbor Check

Python

Run

Avoid repeated node.

Push Edge into Heap

Python

Run

Adds edge for future selection.

Print Total Cost

Python

Run

Displays final MST cost.

Graph Representation

```
graph = {
    'A': [('B', 2), ('C', 6), ('D', 3)],
    'B': [('A', 2), ('D', 5)],
    'C': [('A', 6), ('D', 4)],
```

Python

Run

Weighted undirected graph.

Function Call

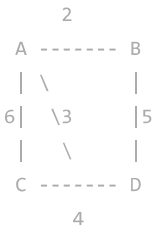
Python

Run

Starts MST construction from A.

5. DRY RUN

Initial Graph



Step 1

Start from A

Visited:

A

Heap:

(2,B), (6,C), (3,D)

Step 2

Select minimum:

A → B = 2

Visited:

A, B

Step 3

Select minimum:

A → D = 3

Visited:

A, B, D

Step 4

Select minimum:

$D \rightarrow C = 4$

Visited:

A, B, C, D

Total MST Cost

$2 + 3 + 4 = 9$

Output

Visited: A Weight: 0
Visited: B Weight: 2
Visited: D Weight: 3
Visited: C Weight: 4

Total MST Cost: 9

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is Prim’s Algorithm?

A greedy algorithm to find MST.

2. What is MST?

Minimum Spanning Tree.

3. What is spanning tree?

Tree connecting all vertices.

4. How many edges in MST?

$V - 1$ edges.

5. Why Prim’s is greedy?

Chooses minimum edge immediately.

6. Which graph is used?

Weighted undirected graph.

7. What data structure is used?

Priority Queue.

8. Why heapq used?

Efficient minimum edge selection.

9. What is edge weight?

Cost between nodes.

10. What is total MST cost?

Sum of selected edges.

Intermediate Questions

11. Time complexity using matrix?

$O(V^2)$

12. Time complexity using heap?

$O((V+E)\log V)$

13. Space complexity?

$O(V)$

14. Is Prim's optimal?

Yes.

15. Is Prim's complete?

Yes.

16. Why visited set used?

Avoid cycles.

17. Can Prim's work on disconnected graph?

No.

18. What happens if graph disconnected?

MST cannot be formed.

19. What is dense graph?

Graph with many edges.

20. Why Prim's good for dense graph?

Efficient edge expansion.

Advanced Questions

21. Difference between Prim's and Kruskal's?

Prim's is vertex-based.

22. Difference between Prim's and Dijkstra?

Prim's builds MST, Dijkstra shortest path.

23. What is cycle?

Closed path in graph.

24. Why cycles avoided?

Tree cannot contain cycles.

25. Can Prim's start anywhere?

Yes.

26. Does starting node affect cost?

No.

27. Why minimum edge selected?

To minimize total cost.

28. What is greedy choice?

Minimum weight edge.

29. Can negative weights exist?

Yes.

30. Why MST useful?

Reduces network cost.

Tricky External Viva Questions

31. Why starting node weight is 0?

No edge needed initially.

32. Why continue statement used?

Skip already visited nodes.

33. What if equal edge weights exist?

Algorithm still works.

34. Why heap stores tuples?

Stores weight and node together.

35. Why adjacency list used?

Memory efficient representation.

36. What dominates complexity?

Heap operations.

37. What if graph has self-loop?

Ignored if unnecessary.

38. Can Prim's work recursively?

Possible but uncommon.

39. Why graph should be connected?

To include all vertices.

40. What is parent node in MST?

Node from which edge selected.

Important “WHY” Questions

41. Why greedy approach works in MST?

Local minimum edges produce global minimum tree.

42. Why minimum heap used?

Fast minimum edge extraction.

43. Why visited set important?

Prevents cycle formation.

44. Why MST has V-1 edges?

Minimum edges needed to connect vertices.

45. Why Prim’s important in networking?

Minimizes infrastructure cost.

Output Questions

46. What is total MST cost?

9

47. Which node visited first?

A

48. Which edge selected first?

A → B = 2

49. Which edge selected second?

A → D = 3

50. Why Prim’s widely used?

Efficient minimum network construction.

📄 ↺ ↻ ⋮

P3(Kruskal).py
Python

Problem Statement

Implement Greedy Search Algorithm for Minimum Spanning Tree using Kruskal's Algorithm.

1. THEORY

Introduction to Greedy Algorithm

A Greedy Algorithm:

- Makes the best possible choice at current step
- Never revisits previous decisions
- Attempts to produce globally optimal solution

What is Minimum Spanning Tree (MST)?

A Minimum Spanning Tree is:

- A subset of graph edges
- Connects all vertices
- Has minimum total edge weight
- Contains no cycles

Conditions of MST

For:

- V vertices

MST contains:

- $V - 1$ edges

What is Kruskal's Algorithm?

Kruskal's Algorithm is a Greedy Algorithm used to:

- Find Minimum Spanning Tree of weighted graph.

It works by:

1. Sorting edges by weight
2. Selecting smallest edge
3. Avoiding cycles

Why Kruskal's Algorithm is Greedy?

Because:

- At every step it chooses smallest available edge.

This immediate best choice makes it greedy.

Working Principle

1. Sort all edges
2. Pick smallest edge
3. Check cycle formation
4. If no cycle:
 - Add edge to MST
5. Repeat until MST complete

Cycle in Graph

A cycle means:

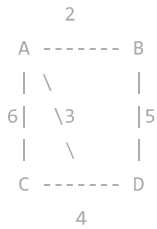
- Closed path in graph

Example:

$A \rightarrow B \rightarrow C \rightarrow A$

MST must not contain cycles.

Example Graph



Sorted Edges

Edge	Weight
A-B	2
A-D	3
C-D	4
B-D	5
A-C	6

MST Construction

Select:

- A-B = 2
- A-D = 3
- C-D = 4

Total Cost:

$2 + 3 + 4 = 9$

Applications of Kruskal’s Algorithm

- Network design
- Telephone wiring
- Road planning
- Computer networking
- Electrical circuits

Advantages of Kruskal’s Algorithm

- Simple implementation
- Efficient for sparse graphs
- Produces optimal MST
- Greedy optimization

Disadvantages of Kruskal’s Algorithm

- Sorting overhead
- Cycle checking required
- Slightly slower for dense graphs

Characteristics of Kruskal’s Algorithm

Feature	Description
Type	Greedy
Problem Solved	MST
Graph Type	Weighted Undirected

Feature	Description
Main Operation	Edge Selection

Union-Find Data Structure

Kruskal's Algorithm uses:

Disjoint Set / Union-Find

It helps:

- Detect cycles efficiently.

Important Operations

Find

Finds parent/root of node.

Union

Combines two sets.

Time Complexity

Sorting Edges

$O(E \log E)$

Union-Find Operations

Almost constant time.

Overall Complexity

$O(E \log E)$

Where:

- E = edges

Space Complexity

$O(V)$

Completeness and Optimality

Property	Kruskal's
Complete	Yes
Optimal	Yes

Difference Between Prim's and Kruskal's

Prim's	Kruskal's
Vertex based	Edge based
Starts from node	Starts from edges
Better for dense graphs	Better for sparse graphs

2. ALGORITHM

Kruskal’s Algorithm

Step-by-Step Algorithm

1. Start
2. Sort all edges in increasing order
3. Create separate set for each vertex
4. Select smallest edge
5. Check whether cycle forms
6. If no cycle:
 - Add edge to MST
 - Perform union
7. Repeat until MST has $V-1$ edges
8. Print MST and total cost
9. Stop

Easy Memory Trick

Sort → Pick Minimum Edge → Avoid Cycle → Repeat

3. CODE

```
# Kruskal's Algorithm

# Find function
def find(parent, i):

    if parent[i] == i:
        return i

    return find(parent, parent[i])

# Union function
def union(parent, rank, x, y):

    root_x = find(parent, x)
    root_y = find(parent, y)

    if rank[root_x] < rank[root_y]:
        parent[root_x] = root_y

    elif rank[root_x] > rank[root_y]:
        parent[root_y] = root_x

    else:
        parent[root_y] = root_x
        rank[root_x] += 1

# Kruskal Algorithm
def kruskal(vertices, edges):

    # Sort edges by weight
    edges = sorted(edges, key=lambda item: item[2])

    parent = []
    rank = []

    # Create disjoint sets
    for node in range(vertices):
        parent.append(node)
        rank.append(0)

    mst = []
    total_cost = 0

    for u, v, weight in edges:

        x = find(parent, u)
        y = find(parent, v)

        # Avoid cycle
        if x != y:

            mst.append((u, v, weight))
            total_cost += weight
```

```
        union(parent, rank, x, y)

    print("Edges in MST:")

    for u, v, weight in mst:
        print(u, "-", v, "=", weight)

    print("Total MST Cost:", total_cost)

# Number of vertices
vertices = 4

# Edge list (u, v, weight)
edges = [
    (0, 1, 2),
    (0, 2, 6),
    (0, 3, 3),
    (1, 3, 5),
    (2, 3, 4)
]
```

Python

Run

4. LINE BY LINE EXPLANATION

Find Function

Python

Run

Finds root parent of node.

Check Root

Python

Run

If node itself is parent:

- Return node.

Recursive Find

Python

Run

Moves upward in tree.

Union Function

Python

Run

Combines two sets.

Find Roots

Python

Run

Gets parent roots.

Rank Comparison

Python	Run
Attach smaller tree below larger tree.	

Parent Update

Python	Run
Changes root parent.	

Equal Rank Case

Python	Run
Increase tree height.	

Kruskal Function

Python	Run
Main MST function.	

Sort Edges

Python	Run
Sort by edge weight.	

Create Parent List

Python	Run
Stores parent of vertices.	

Create Rank List

Python	Run
Stores tree depth.	

Disjoint Set Creation

Python	Run
Create separate set for every vertex.	

Add Parent

Python	Run
Initially each node is own parent.	

Add Rank

Python	Run
Initial rank is zero.	

MST Storage

Python	Run
Stores selected MST edges.	

Total Cost

Python	Run
Stores MST weight.	

Traverse Sorted Edges

Python	Run
Check every edge.	

Find Parents

Python	Run
Find connected components.	

Cycle Check

Python	Run
If roots differ:	
No cycle exists.	

Add Edge

Python	Run
Insert edge into MST.	

Add Cost

Python

Update MST cost.

Run

Perform Union

Python

Merge sets.

Run

Print MST

Python

Displays MST edges.

Run

Print Each Edge

Python

Traverse MST edges.

Run

Print Edge

Python

Displays edge and cost.

Run

Print Total Cost

Python

Displays final MST cost.

Run

Graph Definition

Python

Total vertices.

Run

Edge List

```
edges = [  
    (0, 1, 2),  
    (0, 2, 6),  
    (0, 3, 3),  
    (1, 3, 5),
```

Run

Python

Run

Weighted graph edges.

Function Call

Python

Run

Starts algorithm.

5. DRY RUN

Sorted Edges

Edge	Weight
0-1	2
0-3	3
2-3	4
1-3	5
0-2	6

Step 1

Choose:

$0-1 = 2$

No cycle.

Step 2

Choose:

$0-3 = 3$

No cycle.

Step 3

Choose:

$2-3 = 4$

No cycle.

MST Complete

Edges:

- 0-1
- 0-3
- 2-3

Total Cost

$2 + 3 + 4 = 9$

Output

```
Edges in MST:
0 - 1 = 2
0 - 3 = 3
2 - 3 = 4

Total MST Cost: 9
```

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is Kruskal's Algorithm?

A greedy algorithm to find MST.

2. What is MST?

Minimum Spanning Tree.

3. Why Kruskal's is greedy?

Chooses minimum edge immediately.

4. What type of graph is used?

Weighted undirected graph.

5. What is cycle?

Closed path in graph.

6. Why cycles avoided?

Tree cannot contain cycles.

7. What data structure is used?

Union-Find.

8. What is disjoint set?

Collection of separate sets.

9. What is union operation?

Combines sets.

10. What is find operation?

Finds root parent.

Intermediate Questions

11. Time complexity?

$O(E \log E)$

12. Space complexity?

$O(V)$

13. Is Kruskal optimal?

Yes.

14. Is Kruskal complete?

Yes.

15. Why edges sorted?

To select minimum edges first.

16. What is rank?

Tree depth measure.

17. Why rank used?

Reduce tree height.

18. What is sparse graph?

Graph with fewer edges.

19. Why Kruskal good for sparse graphs?

Processes edges efficiently.

20. What is greedy choice?

Minimum weight edge.

Advanced Questions

21. Difference between Prim's and Kruskal's?

Prim's vertex-based, Kruskal edge-based.

22. Can Kruskal work on disconnected graph?

Produces forest.

23. What is spanning forest?

Collection of MSTs.

24. Why union-find important?

Efficient cycle detection.

25. Can Kruskal handle negative weights?

Yes.

26. Why MST has $V-1$ edges?

Minimum edges needed for connectivity.

27. What happens if equal weights exist?

Still works correctly.

28. What dominates complexity?

Sorting.

29. Why recursive find used?

Trace parent root.

30. Can adjacency matrix be used?

Yes.

Tricky External Viva Questions

31. Why parent initialized to itself?

Initially separate sets.

32. Why continue cycle checking?

Prevent invalid MST.

33. Why sorted() uses lambda?

Sort by edge weight.

34. What if graph has cycle initially?

Algorithm removes unnecessary edges.

35. Why rank optimization important?

Improves efficiency.

36. What if all edges same weight?

Any valid MST possible.

37. Can Kruskal be recursive?

Union-find uses recursion.

38. Difference between tree and graph?

Tree has no cycles.

39. Can MST exist in directed graph?

Typically no.

40. What is connected graph?

Every node reachable.

Important “WHY” Questions

41. Why minimum edge selected first?

To minimize total MST cost.

42. Why cycle checking necessary?

MST cannot contain cycles.

43. Why greedy approach works?

Local minimum edges build global minimum tree.

44. Why union operation required?

Merge connected components.

45. Why Kruskal important in networking?

Minimizes infrastructure cost.

Output Questions

46. What is total MST cost?

9

47. Which edge selected first?

0-1 = 2

48. Which edge selected second?

0-3 = 3

49. Which edge rejected due to cycle?

1-3 = 5

50. Why Kruskal widely used?

Simple and efficient MST algorithm.

📄 ↺ ↻ ⋮

4. Implement a solution for a Constraint Satisfaction Problem using Branch and Bound and Backtracking for n-queens problem or a graph coloring problem.

P4(Backtracking).py

Python

P4(Branch & Bound).py

Python

Perfect.

Experiment 4 contains two important concepts:

- 1. Backtracking
- 2. Branch and Bound

And the problem used is:

N-Queens Problem

We will go one by one because both are major viva topics.

First We'll Start With:

N-Queens using Backtracking

After that we'll do:

N-Queens using Branch and Bound

For each one, I'll give:

- Complete theory
- Constraint Satisfaction Problem explanation
- N-Queens concept
- Why Backtracking / Branch & Bound is used
- Full algorithm
- Complete code explanation
- Dry run
- Time & space complexity
- Completeness & optimality
- 40–50 viva questions with answers
- External viva questions

Now please say:

- **“Start Backtracking”**

Then I'll begin with the full detailed explanation.



P4(Backtracking).py
Python

P4(Branch & Bound).py
Python

Experiment No. 4 (I)

N-Queens Problem using Backtracking

Problem Statement

Implement a solution for Constraint Satisfaction Problem using Backtracking for N-Queens Problem.

1. THEORY

What is Constraint Satisfaction Problem (CSP)?

A Constraint Satisfaction Problem is a problem where:

- We must satisfy given conditions (constraints)
- While finding a valid solution

Components of CSP

Component	Meaning
Variables	Items to assign
Domain	Possible values
Constraints	Rules to satisfy

Examples of CSP

1. Sudoku
2. N-Queens Problem
3. Graph Coloring
4. Timetable Scheduling

What is N-Queens Problem?

The N-Queens problem means:

- Place N queens on an N×N chessboard
- Such that no two queens attack each other

Queen’s Attack Rules

A queen can attack:

1. Horizontally
2. Vertically
3. Diagonally

So:

- No two queens can share:
 - Same row
 - Same column
 - Same diagonal

Example of 4-Queens Problem

We must place:

- 4 queens on 4×4 board

Valid Solution:

```
. Q . .
. . . Q
Q . . .
. . Q .
```

Where:

- Q = Queen
- . = Empty space

What is Backtracking?

Backtracking is a problem-solving technique where:

1. Try a solution
2. Check validity
3. If invalid:
 - Undo choice
 - Try another option

It is:

- Recursive
- Depth-first approach

Why Backtracking for N-Queens?

Because:

- Many placements become invalid
- We need to:
 - Try
 - Check
 - Backtrack if wrong

Working Principle of Backtracking

1. Place queen
2. Check safety
3. Move to next row
4. If no valid position:
 - Remove queen
 - Go back
5. Try next column

Backtracking Concept

Try → Check → Fail → Go Back → Retry

Applications of Backtracking

- 1. Sudoku Solver
- 2. Maze Solving
- 3. N-Queens
- 4. Password Generation
- 5. Graph Coloring

Advantages of Backtracking

- 1. Systematic search
- 2. Finds all solutions
- 3. Memory efficient
- 4. Recursive simplicity

Disadvantages of Backtracking

- 1. Slow for large inputs
- 2. Exponential time complexity
- 3. Many recursive calls

Time Complexity

Worst Case:

$O(N!)$

Because many queen arrangements are checked.

Space Complexity

$O(N^2)$

For storing chessboard.

Completeness and Optimality

Property	Backtracking
Complete	Yes
Optimal	No

Difference Between Backtracking and Branch & Bound

Backtracking	Branch & Bound
Feasibility based	Cost based
Removes invalid solutions	Removes non-optimal solutions
DFS style	BFS/Best-first style

2. ALGORITHM

N-Queens using Backtracking Algorithm

Step-by-Step Algorithm

- 1. Start
- 2. Place queen in first row
- 3. Check if position is safe
- 4. If safe:
 - Place queen
 - Move to next row

5. If unsafe:
- Try next column
6. If all columns fail:
- Remove previous queen

• Backtrack
7. Repeat until all queens placed
8. Print solution
9. Stop

Easy Memory Trick

Place → Check → Backtrack → Retry

3. CODE

```
# N-Queens using Backtracking

N = 4

# Print Board
def print_board(board):

    for row in board:
        print(" ".join(row))

    print()

# Check Safe Position
def is_safe(board, row, col):

    # Check column
    for i in range(row):
        if board[i][col] == 'Q':
            return False

    # Check left diagonal
    i, j = row - 1, col - 1

    while i >= 0 and j >= 0:
        if board[i][j] == 'Q':
            return False
        i -= 1
        j -= 1

    # Check right diagonal
    i, j = row - 1, col + 1

    while i >= 0 and j < N:
        if board[i][j] == 'Q':
            return False
        i -= 1
        j += 1

    return True

# Solve Function
def solve(board, row):

    # Base Case
    if row == N:
        print_board(board)
        return

    # Try all columns
    for col in range(N):

        if is_safe(board, row, col):

            board[row][col] = 'Q'

            solve(board, row + 1)

            # Backtrack
            board[row][col] = '.'

# Driver Code
board = [['.' for _ in range(N)] for _ in range(N)]
```

Python

```
solve(board, 0)
```

Run

4. LINE BY LINE EXPLANATION

Board Size

Python

Defines:

- 4×4 chessboard.

Run

Print Board Function

Python

Prints final solution board.

Run

Traverse Rows

Python

Access each row.

Run

Print Row

Python

Displays board neatly.

Run

Safety Function

Python

Checks whether queen placement is valid.

Run

Column Check

Python

Checks upper rows in same column.

Run

Queen Exists Check

Python

If queen already exists:

- Position unsafe.

Run

Left Diagonal

Python

Run

Move diagonally upward left.

Left Diagonal Loop

Python

Run

Continue until boundary reached.

Left Diagonal Queen Check

Python

Run

Unsafe if queen found.

Right Diagonal

Python

Run

Move diagonally upward right.

Right Diagonal Loop

Python

Run

Continue until board boundary.

Right Diagonal Queen Check

Python

Run

Unsafe if queen found.

Safe Return

Python

Run

Position safe.

Solve Function

Python

Run

Recursive backtracking function.

Base Case

Python

Run

All queens placed successfully.

Print Solution

Python

Run

Displays valid arrangement.

Try All Columns

Python

Run

Check every column.

Safe Position Check

Python

Run

Valid queen placement check.

Place Queen

Python

Run

Place queen.

Recursive Call

Python

Run

Move to next row.

Backtracking Step

Python

Run

Remove queen if solution fails.

Board Creation

Python

Run

Creates empty chessboard.

Function Call

Python

Run

Starts solving from row 0.

5. DRY RUN

Initial Empty Board

```
. . . .  
. . . .  
. . . .  
. . . .
```

Step 1

Place queen at:

```
(0,1)
```

Step 2

Place queen at:

```
(1,3)
```

Step 3

Place queen at:

```
(2,0)
```

Step 4

Place queen at:

```
(3,2)
```

Final Solution

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

Output

```
. Q . .  
. . . Q  
Q . . .  
. . Q .
```

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is N-Queens Problem?

Place N queens safely on chessboard.

2. What is Backtracking?

Try-and-retry recursive technique.

3. Why Backtracking used in N-Queens?

To remove invalid queen placements.

4. What is CSP?

Constraint Satisfaction Problem.

5. What are constraints in N-Queens?

No same row, column, diagonal.

6. What is recursive function?

Function calling itself.

7. What is base case?

Condition stopping recursion.

8. Why queens attack diagonally checked?

Queens move diagonally.

9. What is safe position?

Position without attacks.

10. What is backtracking step?

Undo previous choice.

Intermediate Questions

11. Time complexity?

$O(N!)$

12. Space complexity?

$O(N^2)$

13. Is Backtracking complete?

Yes.

14. Is Backtracking optimal?

No.

15. Why recursion used?

Natural DFS exploration.

16. Why board initialized with '.'?

Represents empty cell.

17. Why only upper rows checked?

Queens placed row by row.

18. What happens if unsafe position found?

Try next column.

19. Why diagonal checking important?

Queens attack diagonally.

20. Can multiple solutions exist?

Yes.

Advanced Questions

21. What is DFS in Backtracking?

Depth-first exploration.

22. Can N-Queens solved iteratively?

Possible but complex.

23. Why complexity exponential?

Many possible arrangements.

24. What is pruning?

Removing invalid paths early.

25. Is Backtracking brute force?

Optimized brute force.

26. Can Backtracking solve Sudoku?

Yes.

27. Why recursion stack important?

Stores previous states.

28. What if N=1?

One valid solution exists.

29. What if N=2 or N=3?

No valid solution.

30. Why row-wise placement used?

Avoid same-row conflicts.

Tricky External Viva Questions

31. Why columns checked separately?

Queens attack vertically.

32. Why lower rows not checked?

Not yet filled.

33. What if queen placed incorrectly?

Backtracking removes it.

34. What dominates complexity?

Recursive branching.

35. Why recursion preferred?

Simplifies implementation.

36. Can arrays optimize checking?

Yes.

37. Why all columns tried?

Search all possibilities.

38. What is dead state?

No valid future placement.

39. What is state space tree?

Tree of possible solutions.

40. Can Backtracking guarantee solution?

Yes if solution exists.

Important “WHY” Questions

41. Why Backtracking efficient?

Invalid paths removed early.

42. Why queen removed after recursion?

To try alternate solutions.

43. Why only previous rows checked?

Future rows empty.

44. Why N-Queens important in AI?

Classic CSP problem.

45. Why recursion suitable here?
Natural exploration of possibilities.

Output Questions

46. What does Q represent?
Queen.

47. What does '.' represent?
Empty cell.

48. How many queens for N=4?
4 queens.

49. Which queen placed first?
(0,1) in shown solution.

50. Why output valid?
No queens attack each other.

📄 ↺ ↻ ⋮

| 5. | Develop an elementary chatbot for any suitable customer interaction application.

For Experiment 5:

Develop an elementary chatbot for any suitable customer interaction application.

I need the chatbot code file (or paste the code here) because chatbot implementations can differ:

- menu-based chatbot,
- rule-based chatbot,
- keyword chatbot,
- AI chatbot,
- customer support chatbot, etc.

Once you upload the code, I'll give you:

- Complete chatbot theory
- What is NLP and AI chatbot
- Types of chatbots
- Rule-based chatbot explanation
- Algorithm
- Full line-by-line code explanation
- Dry run / conversation flow
- Advantages & disadvantages
- Applications
- Time & space complexity (if applicable)
- 40–50 viva questions with answers
- Tricky external viva questions
- Important interview/viva terminology

Upload the Experiment 5 code and we'll start.

📄 ↺ ↻ ⋮

AI_EXP_5.py
Python

go

Experiment No. 5

Elementary Chatbot for Customer Interaction

Problem Statement

Develop an elementary chatbot for any suitable customer interaction application.

1. THEORY

What is a Chatbot?

A Chatbot is a software application that:

- Simulates human conversation
- Interacts with users automatically
- Responds to user queries

It works through:

- Text messages
- Voice commands

Purpose of Chatbot

The main purpose of chatbot is:

- Automated communication
- Customer support
- Faster response handling

Examples of Chatbots

1. Customer Support Bots
2. Banking Assistants
3. AI Assistants
4. E-commerce Bots
5. Food Ordering Bots

Examples:

- Siri
- Alexa
- ChatGPT
- Google Assistant

Types of Chatbots

1. Rule-Based Chatbot

Works using predefined rules and conditions.

Example:

```
If user says "hello"  
→ Reply "Hi"
```

This experiment uses:

Rule-Based Chatbot

2. AI Chatbot

Uses:

- Artificial Intelligence
- Machine Learning
- NLP

More intelligent and dynamic.

What is NLP?

Natural Language Processing

A field of AI that helps computer:

- Understand human language
- Process text/speech
- Generate responses

How Rule-Based Chatbot Works

1. User enters message
2. Chatbot compares keywords
3. Matching condition executes
4. Response displayed

Features of Elementary Chatbot

1. Simple interaction
2. Keyword matching
3. Predefined responses
4. Fast communication
5. Beginner-friendly

Applications of Chatbots

1. Customer Support
2. Banking
3. E-commerce
4. Healthcare
5. Education
6. Food Ordering
7. Hotel Booking

Advantages of Chatbots

1. 24×7 availability
2. Fast responses
3. Reduces human workload
4. Cost-effective
5. Easy interaction

Disadvantages of Chatbots

1. Limited understanding
2. Cannot handle complex queries
3. No emotions
4. Rule-based limitations

Working of Chatbot



Characteristics of Rule-Based Chatbot

Feature	Description
Intelligence	Low
Learning	No
Speed	Fast
Complexity	Simple

Time Complexity

Depends on:

- Number of conditions

Approximate complexity:

$O(n)$

Where:

- n = number of conditions checked.

Space Complexity

$O(1)$

Completeness and Optimality

Property	Chatbot
Complete	No
Optimal	No

Because:

- Only predefined responses exist.

Difference Between Rule-Based and AI Chatbot

Rule-Based	AI Chatbot
Predefined rules	Learns dynamically
Simple	Intelligent
No learning	Machine learning
Limited responses	Flexible responses

2. ALGORITHM

Chatbot Algorithm

Step-by-Step Algorithm

1. Start
2. Display welcome message
3. Take user input
4. Convert input to lowercase
5. Check conditions:
 - hello
 - bye
 - help
 - etc.
6. Display matching response
7. Repeat until user exits
8. Stop

Easy Memory Trick

Input → Match Keyword → Respond

3. CODE

```
# Elementary Chatbot

print("Chatbot: Hello! Type 'bye' to exit.")

while True:

    user = input("You: ").lower()

    # Greetings
    if user == "hello" or user == "hi":
        print("Chatbot: Hello! How can I help you?")

    # Asking Name
    elif user == "your name":
        print("Chatbot: I am a simple chatbot.")

    # Help Query
    elif user == "help":
        print("Chatbot: I can answer simple questions.")

    # Exit Condition
    elif user == "bye":
        print("Chatbot: Goodbye!")
        break

    # Unknown Input

Python Run
```

4. LINE BY LINE EXPLANATION

Welcome Message

```
Python Run
```

Displays chatbot greeting.

Infinite Loop

```
Python Run
```

Keeps chatbot running continuously.

User Input

```
Python Run
```

Takes user message.

`.lower()` converts text into lowercase.

Example:

```
HELLO → hello
```

Greeting Condition

```
Python Run
```

Checks greeting words.

Greeting Response

Python	Run
Displays response.	

Name Query

Python	Run
Checks if user asks chatbot name.	

Name Response

Python	Run
Displays chatbot introduction.	

Help Query

Python	Run
Checks help request.	

Help Response

Python	Run
Displays help information.	

Exit Condition

Python	Run
Checks termination command.	

Exit Message

Python	Run
Displays exit message.	

Break Statement

Python	Run
Stops loop and exits chatbot.	

Unknown Input

Python	Run
Handles unmatched queries.	

Unknown Response

Python	Run
Default reply.	

5. DRY RUN

Program Start

Chatbot: Hello! Type 'bye' to exit.

User Input 1

You: hello

Condition matched:

hello

Output:

Chatbot: Hello! How can I help you?

User Input 2

You: your name

Output:

Chatbot: I am a simple chatbot.

User Input 3

You: help

Output:

Chatbot: I can answer simple questions.

User Input 4

You: bye

Output:

Chatbot: Goodbye!

Program terminates.

6. IMPORTANT VIVA QUESTIONS WITH ANSWERS

Basic Questions

1. What is a chatbot?

Software that simulates conversation.

2. What is elementary chatbot?

Simple rule-based chatbot.

3. What is NLP?

Natural Language Processing.

4. Why chatbot used?

Automated communication.

5. Which type of chatbot used here?

Rule-based chatbot.

6. What is keyword matching?

Checking specific words in input.

7. Why `.lower()` used?

Case-insensitive matching.

8. What does `while True` do?

Creates infinite loop.

9. What is break statement?

Stops loop execution.

10. What is user input?

Message entered by user.

Intermediate Questions

11. Can chatbot learn automatically?

Not this rule-based chatbot.

12. What is AI chatbot?

Chatbot using AI and ML.

13. Difference between AI and rule-based chatbot?

AI learns dynamically.

14. What is customer interaction system?

System communicating with users.

15. Why conditions used?

To match queries.

16. What is default response?

Response for unknown query.

17. Can chatbot answer all questions?

No.

18. Why rule-based chatbot limited?

Works only on predefined rules.

19. What is conversational AI?

AI-based communication system.

20. What is chatbot workflow?

Input → Processing → Response.

Advanced Questions

21. Time complexity of chatbot?

$O(n)$

22. Space complexity?

$O(1)$

23. What is intent recognition?

Identifying user purpose.

24. Can chatbot use database?

Yes.

25. Can chatbot use APIs?

Yes.

26. Can chatbot handle voice?

Yes with speech processing.

27. Why chatbot popular?

Fast automated responses.

28. Can chatbot replace humans?

Partially.

29. What is machine learning chatbot?

Bot that learns from data.

30. What is conversational flow?

Sequence of interaction.

Tricky External Viva Questions

31. Why lowercase conversion important?

Simplifies comparisons.

32. Why infinite loop used?

Continuous conversation.

33. What if break removed?

Program never exits.

34. Why else block important?

Handles unknown inputs.

35. Can chatbot be multilingual?

Yes.

36. What if user enters random text?

Default response shown.

37. Why chatbot called intelligent system?

Automated interaction capability.

38. Can chatbot integrate with websites?

Yes.

39. What is chatbot limitation?

No true understanding.

40. Difference between virtual assistant and chatbot?

Virtual assistant more intelligent.

Important “WHY” Questions

41. Why chatbot important in customer service?

Quick automated support.

42. Why rule-based chatbot easy?

Simple condition matching.

43. Why input converted to lowercase?

Avoid case mismatch.

44. Why break statement necessary?

Exit chatbot safely.

45. Why default response required?

Handle unmatched inputs.

Output Questions

46. What happens when user types hello?

Greeting displayed.

47. What happens when user types bye?

Program exits.

48. What happens for unknown query?

Sorry message shown.

49. Which condition checked first?

Greeting condition.

50. Why chatbot useful in companies?

Reduces customer support workload.

📄 ↺ ↻ ⋮

how can i export this chat so claude can access it and then use it ass input to create a proper pdf of it

You have several easy ways to export this chat and give it to Claude for creating a proper PDF.

Method 1 — Best and Simplest (Recommended)

Export ChatGPT Conversation

On Desktop Web

- 1. Open this chat
- 2. Click the **Share** button (top-right)
- 3. Click **Create Link**
- 4. Copy the shared link

Then:

- Open Claude
- Paste the link
- Ask:

Create a clean professional PDF study guide from this conversation.

Requirements:

- Proper headings
- Table formatting
- Syntax-highlighted code
- Page numbers
- Viva questions in separate sections
- Attractive academic formatting

Edit

Claude can usually read the shared conversation directly.

Method 2 — Copy-Paste into Claude

If sharing doesn't work:

1. Select the entire chat
2. Copy it
3. Paste into Claude

Then ask Claude:

Convert this into a professional academic PDF with:

- Proper hierarchy
- Clean formatting
- Theory sections
- Algorithms
- Code blocks

Edit

- Tables
- Page numbers
- Colored headings

Method 3 — Export Entire ChatGPT Data

Full Export

1. Click your profile picture
2. Go to **Settings**
3. Open **Data Controls**
4. Click **Export Data**
5. Download ZIP file from email

Inside ZIP:

- Conversations are stored in HTML/JSON format.

You can upload those files to Claude.

Method 4 — Save as PDF Directly (Fastest)

Chrome / Edge Browser

1. Press:

Ctrl + P

2. Choose:

Save as PDF

3. Save file

Then upload that PDF to Claude and ask it to:

- restructure,
- beautify,
- organize notes.

Best Prompt to Use in Claude

Paste this:

Edit

Create a highly professional academic PDF from these AI practical notes.

Requirements:

- Professional textbook formatting
- Proper headings and hierarchy
- Colored section titles
- Clean tables
- Syntax-highlighted code
- Proper spacing
- Viva questions in separate sections
- Add page numbers
- Add table of contents
- Make it easy for practical viva preparation
- Keep all content detailed
- Do not remove explanations
- Make it print-ready

My Recommendation

Best workflow:



Claude is particularly good at:

- document formatting,
- PDF structuring,
- beautiful layouts.

While this chat is already strong in:

- technical depth,
- viva preparation,
- detailed explanations.

